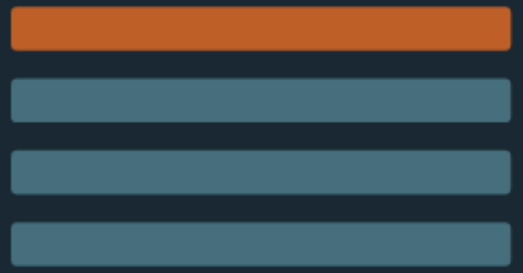
MAESTRÍA · ESPACIO ACADÉMICO

Gestión de Configuración y Mantenimiento de Software



Sesión 1 — Unidad 1 · Evolución del software y mantenimiento

Lo que nadie quiere tocar también se deteriora.

César Augusto Vega Fernández

Encuentro sincrónico · Leyes de Lehman · ISO/IEC/IEEE 14764 · Deuda técnica

EL RETO DE LA UNIDAD 1 · ACTIVIDADES EN LA PLATAFORMA

Un diagnóstico antes de controlar el cambio

«Este sistema funciona, pero cada cambio cuesta más que el anterior y cada despliegue da más miedo.

Antes de decidir cómo controlar sus cambios necesitamos un diagnóstico: cómo ha evolucionado, en qué se va el esfuerzo de mantenimiento, qué deuda arrastra y qué riesgos corre.

Con eso diseñamos lo que sigue.»

Responsable del sistema que eligió cada grupo

Actividad 0 · Presentación del reto y contexto del sistema

· Formativa · 6 horas · grupal, de 2 a 3

· Documento de 2 a 3 páginas o presentación de 5 a 7 diapositivas: sistema, caracterización, problema inicial, reto, alcance y criterios de éxito

Actividad 1 · Análisis de evolución, deuda técnica y mantenimiento

· 15 % · 12 horas · grupal, de 2 a 3

· Documento técnico de 5 a 7 páginas, en APA 7

· Problemas de evolución · deuda técnica · tipos de mantenimiento

· Riesgos: cambios no controlados y falta de estrategia de evolución

El sistema que elijan hoy los acompaña todo el curso: en la Unidad 2 le diseñan la gestión de configuración, en la 3 el control de versiones y el pipeline, y en la 4 las mejoras.

Los F-nn son los formatos del curso, las plantillas que preparan cada sección del documento. F-01 a F-07 son los de la Unidad 1. No se suben a la plataforma.

CÓMO VAMOS A USAR LA SESIÓN

Primero qué significa, después ejemplos, después ustedes

PRIMERO

Conceptos

Envejecimiento, leyes de Lehman, tipos de mantenimiento, deuda técnica y los riesgos de no gestionar la evolución.

DESPUÉS

Ejemplos

Los formatos llenos con la historia real de Flask, un framework web con quince años de versiones públicas.

Y EL RESTO DE LA SESIÓN

El hacer

El taller en salas: cada grupo elige su sistema y sale con la Actividad 0 en borrador.

El bloque que no se recorta es el tercero. Si los conceptos se alargan, se acortan los ejemplos: están completos en el recorrido resuelto del repositorio.

BLOQUE 1 · EL PROBLEMA

Lo que nadie quiere tocar también se deteriora

Dilatación de tiempos

Cambios que parecían sencillos toman mucho más esfuerzo del previsto.

Inestabilidad operativa

Cada actualización trae más errores, aunque el equipo tenga experiencia.

Silos de conocimiento

El sistema lo entienden pocas personas, que se vuelven indispensables.

El envejecimiento según Parnas (1994)

El software envejece por dos causas: no se adapta a un entorno que cambia, y los cambios que sí se le hacen los hace gente que no entiende el diseño original. A diferencia del hardware, no se desgasta: se deteriora por los efectos de los cambios.

Los tres síntomas sirven como primera evidencia en la Actividad 0: ¿cuáles se ven en su sistema y quién lo puede confirmar?

BLOQUE 1 · POR QUÉ CAMBIA EL SOFTWARE

Tres tipos de programa según Lehman

S · especificado

Resuelve un problema definido formalmente.

Es correcto o no lo es respecto de su especificación.

Ejemplo: una función que ordena una lista.

P · problema

Resuelve un problema real con una aproximación aceptable.

Se juzga por la utilidad de la solución.

Ejemplo: un programa que juega ajedrez.

E · embebido

Opera en el mundo real y forma parte de él.

Su uso cambia el entorno, y el entorno cambiado exige cambiar el programa.

Ejemplo: un sistema bancario, un framework web.

Las leyes de la evolución se formularon para los programas tipo E. Casi todo el software que una organización mantiene es de ese tipo.

Lehman, M. M. (1980). Programs, life cycles, and laws of software evolution · Lehman, M. M. (1996). Laws of software evolution revisited.

BLOQUE 1 · LEYES DE LA EVOLUCIÓN · HERRAMIENTA EN VIVO

Las ocho leyes de Lehman

Ley	Qué dice	Qué evidencia la mostraría
I · Cambio continuo	Un sistema E que no se adapta se vuelve progresivamente menos satisfactorio	Cambios adaptativos durante toda la vida
II · Complejidad creciente	La complejidad crece salvo que se trabaje para reducirla	Trabajo preventivo, retiro de código, o indicadores de complejidad al alza
III · Autorregulación	La evolución se autorregula: sus medidas tienen tendencias estables	Tendencias reconocibles en tamaño y actividad
IV · Estabilidad organizacional	La tasa media de trabajo efectivo es invariante	Cambios por mes estables aunque cambie el equipo
V · Familiaridad	El contenido incremental de cada versión se mantiene acotado	Después de una versión grande, una pequeña
VI · Crecimiento continuo	La funcionalidad debe crecer para mantener la satisfacción	Tamaño creciente versión tras versión
VII · Calidad decreciente	La calidad se percibe en declive si no se mantiene y adapta	Defectos reportados por versión
VIII · Retroalimentación	La evolución es un sistema de retroalimentación de varios niveles	Decisiones que responden a usuarios, equipo y entorno

Lehman (1996). En el aula se recorre el interactivo de las leyes: el control de trabajo preventivo muestra la ley II.

Cuatro tipos de mantenimiento

	Corrección	Mejora
Reactivo	**Correctivo** · repara una falla que ya se manifestó	**Adaptativo** · responde a un cambio del entorno: lenguaje, dependencias, plataforma, normas
Proactivo	**Preventivo** · corrige fallas latentes o reduce la dificultad de cambios futuros	**Perfectivo** · agrega o mejora funcionalidad, rendimiento o usabilidad

Lo que revela la distribución

Un equipo que solo hace correctivo no tiene espacio para otra cosa.

El perfectivo y el preventivo son los que más se postergan.

La distribución real dice más de la madurez que el proceso declarado.

El tipo se decide leyendo el cambio. El mensaje del commit o del ticket orienta, pero «fix» aparece en correcciones de documentación y en funcionalidades nuevas.

ISO/IEC/IEEE 14764:2022 · Pressman (2010). En el aula se usa el clasificador interactivo: trece cambios reales de Flask para clasificar antes de ver la respuesta.

BLOQUE 1 · LO QUE SE ACUMULA

Deuda técnica: el costo que se paga después

	Imprudente	Prudente
Deliberada	«No hay tiempo para diseñar»	«Hay que publicar ya y asumimos las consecuencias»
Inadvertida	«¿Qué es una arquitectura por capas?»	«Ahora sabemos cómo debimos hacerlo»

Los términos

Interés: lo que cuesta cada cambio mientras la deuda sigue.

Pago: lo que cuesta quitarla.

La deuda que preocupa es la que nadie registra ni cuantifica.

Knight Capital, 2012

Un despliegue dejó uno de ocho servidores con código antiguo, que reactivó una función en desuso desde hacía años. La firma perdió cerca de 460 millones de dólares en unos 45 minutos (SEC, 2013). Código obsoleto que nadie retiró: deuda no registrada.

Cuadrante de Fowler (2009).

BLOQUE 1 · POR QUÉ IMPORTA

El riesgo de no gestionar la evolución

Cambios no controlados

Nadie sabe con certeza qué versión está en producción ni qué cambió entre una entrega y otra.

Una modificación de configuración rompe el sistema aunque el código no haya cambiado.

Falta de estrategia de evolución

Las adaptaciones se hacen cuando el entorno ya obligó, no antes.

La deuda crece sin registro y la complejidad sube sin trabajo preventivo.

Ariane 5, 1996

El cohete se perdió unos 37 segundos después del lanzamiento. Reutilizaba código del Ariane 4 sin verificar que los nuevos valores de vuelo cupieran en una conversión de datos (Lions, 1996). Lehman lo llama principio de incertidumbre: un sistema puede fallar sin que su código cambie, porque cambian los supuestos.

Estos dos grupos de riesgo son los que pide analizar la Actividad 1. Responderlos es la tarea de las unidades 2 y 3.

BLOQUE 1 · LA CADENA QUE SOSTIENE EL DOCUMENTO

Del sistema a los riesgos, con evidencia

Sistema y reto
F-01 · qué es, para quién, qué se aborda

›

Evidencia
F-02 · historial, versiones, tickets, incidentes

›

Mantenimiento
F-03 · en qué se va el esfuerzo

›

Leyes y deuda
F-04 y F-05 · qué se observa y qué se arrastra

›

Riesgos
F-06 · qué puede salir mal y por qué importa

Cada eslabón se apoya en el anterior. Un riesgo que no se puede rastrear hasta una evidencia del sistema es una opinión.

El reto y el alcance, escritos antes de medir

Caracterización

Tipo: framework web de Python, publicado como biblioteca.

Usuarios: desarrolladores de aplicaciones y autores de extensiones.

Componentes: aplicación, blueprints, línea de comandos, sesiones, JSON y plantillas.

Reto, alcance y criterios

Reto: ¿puede una organización seguir dependiendo de Flask cinco años más, y qué tendría que vigilar?

Alcance: el paquete flask. **Queda fuera:** pruebas, documentación y extensiones.

Criterio de éxito: cada afirmación se rastrea hasta el historial público.

Decidir qué se mide antes de medir

flask.py 0.1 a 0.4	Todo el framework en un solo archivo.
flask/ 0.5 a 1.0	Paquete con varios módulos. Hasta la 0.10 las pruebas viven dentro, en **flask/testsuite/**.
src/flask/ desde 1.1.0	El paquete pasa a la carpeta src/. Las pruebas, la documentación y los ejemplos están fuera.

Un error de medición

Sin acotar las rutas, la versión 0.10 aparece con **59 archivos**.

Midiendo solo el paquete tiene **20**.

La diferencia son pruebas y setup.py: cambia cualquier conclusión sobre el crecimiento.

```
python herramientas/metricas_evolucion.py flask --incluir "flask.py,flask/**/*.py,src/flask/**/*.py"
```

BLOQUE 2 · EJEMPLO GUIADO 3 · F-02

Quince años de Flask en números

Versión	Fecha	Archivos	Líneas	Agregadas / eliminadas	Commits por mes
0.1	2010-04	1	663	—	—
0.5	2010-07	12	1.970	+1.970 / −1.516	125
1.0	2018-04	20	7.712	+2.336 / −1.160	27
2.0.0	2021-05	21	8.254	+2.727 / −2.794	14
2.3.0	2023-04	22	8.799	+658 / −1.216	15
3.1.0	2024-11	24	9.187	+743 / −396	9

Lo que se ve

Hasta la 1.0 crece casi doce veces; después, 19 % en seis años y medio.

La 2.0.0 elimina más de lo que agrega.

Desde 2012, entre 14 y 28 commits por mes.

Fuente: ejemplo_flask/evolucion_por_version.csv, 20 versiones con funcionalidad nueva. Las versiones atípicas se explican con CHANGES.rst en el recorrido resuelto.

BLOQUE 2 · EJEMPLO GUIADO 4 · F-03

En qué se va el esfuerzo de mantenimiento

Tipo	Commits de la muestra	Sobre los que tocan el producto
Fuera del producto	24 de 40	—
Preventivo	5	31 %
Correctivo	4	25 %
Adaptativo	4	25 %
Perfectivo	3	19 %

Dos hallazgos de método

60 % de la muestra no toca el producto: por eso se usa --solo-fuente.

La heurística por palabras clave acierta en **15 de 40**.

La clasificación se hace leyendo el diff.

Muestra aleatoria de 40 commits sin merges, semilla 1. Clasificación manual en ejemplo_flask/muestra_cambios_clasificada.csv.

Los casos difíciles

Commit	La heurística dice	Leyendo el diff	Por qué
Added template_global, fixes #657	correctivo	perfectivo	Método nuevo con pruebas; el issue pedía la funcionalidad
remove deprecated RequestContext.g	adaptativo	preventivo	El entorno no cambió; se retira código propio obsoleto
add SESSION_COOKIE_PARTITIONED config	perfectivo	adaptativo	Los navegadores introdujeron las cookies particionadas
Pass maxsplit via kwarg to re.split	sin sugerencia	adaptativo	Python 3.13 marcó como obsoleto el argumento por posición

Un caso difícil bien razonado vale más en el documento que diez casos fáciles. Anoten por qué admitía dos tipos y cuál eligieron.

Qué leyes se observan en Flask

Ley	Evidencia	Contraevidencia	Veredicto
I · Cambio continuo	4 de 16 cambios al producto son adaptativos	—	Se observa
II · Complejidad	Menciones a «deprecat»: 27, 91 y 2 en 2.0.0, 2.2.0 y 3.0.0	type: ignore pasa de 58 a 105	Con matices
IV · Estabilidad	14 a 28 commits por mes desde 2012, con 13 a 209 autores	9 por mes en la 3.1.0	Se observa
V · Familiaridad	2.0.0 → 2.1.0, 2.2.0 → 2.3.0, 3.0.0 → 3.1.0: grande y luego pequeña	1.0 y 1.1.0 grandes seguidas	Se observa
VI · Crecimiento	De 663 a 9.187 líneas	Casi plano desde 2018	Con matices
VII · Calidad	—	—	Los datos no alcanzan

«Los datos no alcanzan» es una respuesta válida si se dice qué dato haría falta: aquí, la serie de defectos reportados por versión.

Deuda, riesgos y recomendación

F-05 · Deuda vigente

Supresiones type: ignore (105 en 3.1.0, 90 hoy).

Alias y firmas obsoletas durante el período de la 3.2.0.

Despacho interno apoyado en objetos proxy globales.

Soporte de Python 3.9 y 3.10, ya retirado en la 3.2.0.

F-06 · Riesgos para quien depende de Flask

Cambio no controlado: cada versión mayor retira API obsoletas; sin versiones fijadas, una actualización rompe la aplicación.

Falta de estrategia: sin seguir los avisos de obsolescencia, los retiros se descubren el día de actualizar.

Señal a vigilar: ninguna versión con funcionalidad nueva desde noviembre de 2024.

Recomendación: seguir con condiciones. Fijar versiones, seguir los avisos de obsolescencia y vigilar la salida de la 3.2.0.

EL HACER · TALLER EN SALAS · ACTIVIDAD 0

De la sesión salen con su sistema y su reto

1	**Elegir el sistema** Académico, laboral o simulado, revisado contra los criterios del F-01.	8 min
2	**Caracterizarlo** Tipo de sistema, usuarios principales y componentes.	6 min
3	**Identificar el problema inicial** Síntomas de envejecimiento, situaciones de configuración o mantenimiento, riesgos.	6 min
4	**Formular el reto como pregunta** Concreto y relevante para el sistema.	5 min
5	**Definir alcance y criterios de éxito** Qué se aborda, qué queda fuera y cómo sabrán que lo lograron.	5 min

Al salir de la sala: el F-01 en borrador, que es la base de la Actividad 0.

PARA TENER AL LADO DURANTE EL TALLER

Criterios del sistema y comandos

Criterios del sistema (F-01)

El grupo puede modificarlo durante todo el curso

Su código está o puede ponerse en Git

Hay evidencia de su evolución: historial, tickets, incidentes o versiones

No expone datos sensibles, o se pueden anonimizar

Si es laboral, hay autorización para usarlo

Si el sistema tiene historia en Git

git log y **git tag** para ver la historia

python herramientas/metricas_evolucion.py sistema **--incluir** "rutas" **--solo-fuente --semilla 7**

git show commit para leer el diff

Sin historia: la evidencia sale de tickets, incidentes y versiones desplegadas

CIERRE · QUÉ SIGUE

De entender cómo cambia a controlar cómo cambia

ESTA SEMANA

Actividades 0 y 1

Socializar la Actividad 0 para recibir retroalimentación. Después, el análisis de la Actividad 1, verificado con el F-07 antes de entregar.

UNIDAD 2

Gestión de configuración

Qué elementos de su sistema se ponen bajo control, cómo se fijan las líneas base y cómo se decide qué cambio entra. Responde a los riesgos de cambios no controlados.

El diagnóstico de esta unidad dice qué le pasa a un sistema con el tiempo. La gestión de configuración es la disciplina que permite controlarlo.

REFERENCIAS

Fuentes de la sesión

Fowler, M. (2009). Technical debt quadrant. https://martinfowler.com/bliki/TechnicalDebtQuadrant.html

ISO/IEC/IEEE. (2022). ISO/IEC/IEEE 14764:2022 Software engineering — Software life cycle processes — Maintenance.

Lehman, M. M. (1980). Programs, life cycles, and laws of software evolution. Proceedings of the IEEE, 68(9), 1060–1076.

Lehman, M. M. (1996). Laws of software evolution revisited. En C. Montangero (Ed.), Software Process Technology (pp. 108–124). Springer. https://doi.org/10.1007/BFb0017737

Lions, J. L. (1996). Ariane 5 flight 501 failure: Report by the inquiry board. European Space Agency.

Parnas, D. L. (1994). Software aging. En Proceedings of the 16th International Conference on Software Engineering (pp. 279–287). IEEE.

Pressman, R. S. (2010). Software engineering: A practitioner's approach (7.ª ed.). McGraw-Hill.

Securities and Exchange Commission. (2013). In the matter of Knight Capital Americas LLC (Release No. 70694).

Pallets. (2026). Flask [Repositorio de código fuente]. https://github.com/pallets/flask